



EMERITUS

Learn. From the world's best.

Transformer - II



EMERITUS

Learn. From the world's best.

Transformers in Action: Training and Pretraining

Today's Agenda

1

Recap of Async Learnings

Core ideas from earlier sessions

2

Positional Encoding Applications

How it's used in real-world models

3

Live Demonstrations

Four transformer coding examples

4

Recap of Async Learnings

Core ideas from earlier sessions

5

Recap of Async Learnings

Core ideas from earlier sessions

Recap of Async Learning

From Core Concepts to Practical Use

Positional Encoding

Sinusoidal functions add sequence order to transformers for parallel processing with positional awareness

Training Complexity

Computational challenges in transformer training include high memory use, attention complexity scaling, and optimisation strategies

Pre-training & Fine-tuning

Two-stage training covers BERT's masked language modelling, GPT's autoregressive generation, and ViLBERT's multimodal learning

Today's Focus

Turning theory into real, production-ready applications

Applications of Positional Encoding

How Transformers Understand Sequence without Recurrence

Positional encoding helps Transformers understand sequence order without using recurrence, keeping them fast and context-aware.



Language Translation: Keeps word order intact for accurate, natural translations



Speech Recognition: Tracks time-based patterns to distinguish sounds correctly



Genomics Analysis: Interprets DNA sequences by recognising the role of nucleotide positions

Demo 1: Visualising Positional Encoding

Interactive Python Visualisation

Interactive Python demo using NumPy and Matplotlib to plot sinusoidal encodings for different sequence positions and dimensions

```
import numpy as np
import matplotlib.pyplot as plt

def positional_encoding(seq_len, d_model):
    pe = np.zeros((seq_len, d_model))
    for pos in range(seq_len):
        for i in range(0, d_model, 2):
            pe[pos, i] = np.sin(pos / (10000 ** (i / d_model)))
            pe[pos, i + 1] = np.cos(pos / (10000 ** (i / d_model)))
    return pe
```

Demo 1: Visualising Positional Encoding

Interactive Python Visualisation

Key Observations

- Each sequence position has a unique pattern
- Smooth transitions between neighbouring positions
- Early dimensions show high-frequency patterns
- Later dimensions show low-frequency patterns

Applications of Pre-Training and Fine-Tuning

How Models Learn and Adapt



Pre-Training: Learns general language patterns from large unlabeled datasets



Fine-Tuning: Adapts pre-trained models to specific tasks with smaller datasets



Task Specialisation: Custom heads and optimisation improve performance on specialised tasks

Applications of Pre-Training and Fine-Tuning

How Models Learn and Adapt

BERT → Sentiment Analysis

Bidirectional encoding captures subtle sentiment in context

ViLBERT → Visual Question Answering

Combines visual and textual cues for multimodal reasoning

GPT → Text Completion & Dialogue

Autoregressive generation enables coherent, contextually appropriate responses

Demo 2: Text Classification with BERT Fine-Tuning

Sentiment Analysis Using the Transformers Library

Hugging Face Implementation

Showcase BERT's power for text classification, Implement easily with a few lines of Transformers code

```
from transformers import pipeline

# Initialize the sentiment analysis pipeline
classifier = pipeline(
    "sentiment-analysis",
    model="nlpTown/bert-base-multilingual-uncased-sentiment"
)

# Run the classifier on a sample text
result = classifier("This movie was absolutely fantastic!")

# Print the result
print(result)
# [{'label': 'POSITIVE', 'score': 0.9998}]
```

Demo 2: Text Classification with BERT Fine-Tuning

Sentiment Analysis Using the Transformers Library

IMDB Dataset Results

Fine-tuned 50,000 movie reviews, the model reaches 94.2% accuracy in detecting sentiment

- Detects complex language patterns
- Understands context-based sentiment
- Manages negation and sarcasm well
- Adapts easily to new domains

Demo 3: Question Answering with BERT

Extracting Accurate Answers from Text

1

SQuAD Dataset Training: BERT fine-tuned on 100,000+ question-answer pairs from Wikipedia

2

Span Extraction Method: Predicts start and end tokens of the answer within the passage

3

Live Implementation: Demonstrates real-time question input and precise answer extraction

Example Query:

Given a passage on machine learning, ask “What year was the transformer architecture introduced?” and BERT returns **“2017”** with **98.5% confidence**.

GPT Applications

Real-World Uses of Generative Models

Education & Tutoring: Delivers personalised learning, clarifies tough concepts, and adapts to individual learning needs

Conversational AI: Powers chatbots and virtual assistants that maintain context, handle complex queries, and give relevant responses



Code Completion: Helps developers write correct code, suggest improvements, and explain complex logic

Content Generation: Produces consistent, creative, and relevant articles, stories, and marketing copy

Demo 4: GPT Text Generation

1

Prompt Engineering: Create clear prompts to guide the model while keeping responses relevant and creative

2

Temperature Control: Adjust randomness — low (0.2) for focused text, high (0.8) for more varied output

Demo 4: GPT Text Generation

3

Length Parameters: Set limits with *max_length* and stopping rules to control output size

```
from transformers import GPT2LMHeadModel, GPT2Tokenizer

# Load tokenizer and model
tokenizer = GPT2Tokenizer.from_pretrained("gpt2")
model = GPT2LMHeadModel.from_pretrained("gpt2")

# Input prompt
prompt = "The future of artificial intelligence"

# Encode the prompt
inputs = tokenizer.encode(prompt, return_tensors="pt")

# Generate text
generated = model.generate(inputs,
                           max_length=100, temperature=0.7, do_sample=True)
# Decode and print output
output = tokenizer.decode(generated[0], skip_special_tokens=True)
print(output)
```

Output Variations: The same prompt can produce different results, from technical to creative responses

Industry Use Cases

Real-World Impact of AI Across Sectors



Healthcare: Summarises clinical notes, cutting documentation time by 60%, and supports diagnosis with 95% accuracy



Finance: Automates report generation and detects fraud in real time with sub-second responses



Education: Delivers personalised tutoring, adjusting content and teaching style to each learner



Retail: Powers shopping assistants that answer product queries, give tailored suggestions, and manage orders through natural language

Key Takeaways

Core Insights from the Session

Positional Encoding: Helps models understand sequence order while keeping parallel processing and temporal context in language, speech, and genomics

Transfer Learning: Combines pre-training and fine-tuning to power modern NLP and enable quick, high-performance model adaptation

Generative AI: GPT models make large-scale, high-quality text creation accessible for both individual and enterprise use

Q&A Session



Open floor for questions and clarifications.